

ArchGuide

Document Analysis Pipeline

An intelligent rule-based recommender that turns a requirements document into an architecture decision (RAG, Fine-Tuning, CAG, or Hybrid).

Technical Pipeline Bible

Upload to recommendation - every stage, every decision, every fix

PIPELINE AT A GLANCE

Purpose

Decide whether an AI workload should use RAG, Fine-Tuning, CAG, or Hybrid - directly from a requirements document.

Input

PDF, DOCX, or TXT requirements document (max 50 MB). Guest uploads allowed; no authentication required.

Output

Recommended architecture, per-arch suitability scores, factor breakdown, why-not explanations, sensitivity analysis, and a full decision trace.

Determinism

Scoring is fully deterministic - the same signals always produce the same recommendation. Only signal extraction involves an LLM.

Resilience

Three-layer extraction (keyword + LLM + heuristic regex). Pipeline degrades gracefully when Ollama or OpenAI is down.

Audit trail

Every signal carries a `source_text` quote with a page number, verified against the original document text.

Table of Contents

1. End-to-End Flow
2. Upload Endpoint - Validation and Guest Support
3. Document Parsing - PDF, DOCX, TXT
4. Relevance Gates - Why Random PDFs Get Rejected
5. The 10 Architecture Signals
6. Signal Extraction - Three-Layer Strategy
 - 6.1 Layer 1 - Keyword Pre-Extraction
 - 6.2 Layer 2 - LLM Extraction
 - 6.3 Layer 3 - Heuristic Regex Fallback
 - 6.4 The Actual LLM Prompt
7. Vector Indexing and Retrieval
8. Scoring Engine - Rules, Weights, Synergies
 - 8.1 Per-Signal Score Lookup
 - 8.2 Weighted Sum and Normalisation
 - 8.3 Hybrid Synergy Bonus
 - 8.4 CAG Synergy Bonus
 - 8.5 Ranking and Why-Not
 - 8.6 Rules Table Highlights
9. Architecture Recommendations and Confidence
10. Sensitivity Analysis and Stability
11. Persistence - Postgres and Redis
12. Decision Trace and Observability
13. The Journey - Bugs Found and Fixes Applied
 - 13.1 Guest uploads returned generic failure
 - 13.2 Hybrid was mathematically un-winnable
 - 13.3 CAG was losing to FineTuning on bounded corpora
 - 13.4 RAG was penalised on its core strength
 - 13.5 Heuristic extractor was negation-blind
 - 13.6 Volatility regex missed plural feeds
 - 13.7 Bounded-corpus docs had null dataset_size
 - 13.8 Word-order mismatch on queries per day
 - 13.9 Professional-headcount scale unmatched
 - 13.10 Hybrid bonus calibration drift
14. Test Coverage and Verification
15. Operational Cheat Sheet

1. End-to-End Flow

The user uploads a file. The request travels through validation, parsing, relevance gates, vector indexing, signal extraction, scoring, and persistence - in that order. Any stage can abort the request with a structured error. The table below is the complete stage map.

Stage	What happens
1. Upload	POST /api/v1/upload accepts file + provider + optional project_id.
2. Filename validation	Sanitises filename; blocks path traversal (.. and null bytes).
3. File-size validation	Rejects > 50 MB; rejects unsupported MIME types.
4. Session row	Postgres row created with status=processing.
5. Parse	DocumentParser extracts full_text, pages[], word_count.
6. Empty-doc check	Reject if word_count < 10.
7. Relevance gate	Reject random docs (cost reports, screenshots, etc.).
8. Section detection	Find Overview / Data / Latency / etc. headers.
9. Vector indexing	Chunk + embed - FAISS index keyed by session_id.
10. Signal extraction	Keyword scan + LLM (or heuristic fallback).
11. Source verification	Confirm each source_text appears in document.
12. Confidence gate	Reject if < 3 signals have confidence >= 0.35.
13. Concentration warn	Warn if all signals come from a single page.
14. Scoring	Per-signal rules - weighted sum - percentage scores.
15. Synergy bonuses	Hybrid / CAG boost when structural patterns detected.
16. Ranking + why-not	Sort architectures; generate explanations.
17. Sensitivity	Perturb each signal; check recommendation stability.
18. Persist	Result row in Postgres; cache in Redis (24h).
19. Return	AnalysisResponse JSON with the analysis_id.

2. Upload Endpoint - Validation and Guest Support

Route: POST /api/v1/upload. File: backend/app/routers/upload.py. Rate limit: 4 uploads / minute / IP (slowapi).

2.1 Request shape

```
POST /api/v1/upload?provider=ollama&project_id=<uuid>
Content-Type: multipart/form-data
Authorization: Bearer <firebase_token> (OPTIONAL)

file=@<your-document.pdf>
```

2.2 Guest uploads

Authentication is optional. When no Firebase token is sent the upload still succeeds. The Session table has no user_id column so sessions are inherently anonymous-capable. Three cases:

- No project_id sent - the session has no project owner and is accessible to anyone holding the analysis_id.
- project_id pointing to a guest project (user_id begins with guest_) - upload allowed.
- project_id pointing to an authenticated user's project - rejected with HTTP 403.

Why this exists

Until this gate was removed, the endpoint hard-rejected unauthenticated callers with HTTP 401. The frontend rendered that as a generic 'Upload failed' card after the user skipped the auth modal, breaking the guest flow end-to-end.

2.3 Validation layers

- **Filename:** `os.path.basename()` strips directory components; reject if it contains `.` or a null byte.
- **MIME and extension:** only `.pdf`, `.docx`, `.txt` are accepted.
- **Size:** hard limit 50 MB; soft warning at 20 MB.
- **Project ownership:** if a uid is present and `project.user_id` differs, reject with 403. Guest projects (user_id begins with `guest_`) are always allowed.

3. Document Parsing - PDF, DOCX, TXT

File: backend/services/document_parser.py. Each format has its own parser; the output schema is identical regardless of input type:

```
{
  "full_text": "<concatenated body>",
  "pages": [
    {"page_number": 1, "text": "...", "keyword_score": 7},
    ...
  ],
  "word_count": 1284,
  "total_pages": 12
}
```

3.1 Per-format notes

PDF

PyMuPDF (fitz) for fast text extraction. Falls back to OCR via Tesseract for scanned pages with no extractable text.

DOCX

python-docx walks paragraphs; embedded tables are flattened to plain text.

TXT

Read as UTF-8 with errors=replace; the entire body becomes a single page.

3.2 Keyword scoring per page

After parsing, every page is scored by counting how many signal keywords it contains (the keyword list lives in SIGNAL_SCHEMA). Pages with keyword_score < 1 are excluded from the LLM context window because they almost certainly contain no signals worth spending tokens on. This is the single biggest cost saver in the pipeline.

4. Relevance Gates - Why Random PDFs Get Rejected

Three gates fire before any LLM call. Each gate that rejects the document saves real money (LLM tokens) and time.

4.1 Gate 0 - Circular upload detection

If the PDF was generated by ArchGuide itself (a results export), reject immediately. We detect this via a magic-string fingerprint embedded in our PDF metadata. Prevents users from uploading their previous result PDF as a new requirements document.

4.2 Gate 1 - Word count

If `word_count < 80`, reject with HTTP 422. A document this short cannot contain enough signal evidence to make a confident recommendation.

4.3 Gate 2 - Signal category coverage

We tally keyword matches across the 10 signal categories. If fewer than 3 categories have any matches, the document is almost certainly not a requirements specification - it could be a cost report, an invoice, or a company newsletter. Reject with 422 and a precise error message.

4.4 Gate 3 - Post-extraction confidence gate

After signal extraction, if fewer than 3 signals come back with confidence ≥ 0.35 , abort with 422. The document passed the keyword gates but the LLM could not actually find structured requirements inside. Prevents low-quality recommendations from being persisted.

5. The 10 Architecture Signals

Every recommendation reduces to these 10 dimensions. Each signal has a fixed set of allowed values and a weight in the scoring engine. The weight reflects how discriminating that signal is - data_volatility carries the heaviest weight because it is the single biggest split between RAG (handles change) and FineTuning (handles static data).

Signal	Allowed values	Weight	What it captures
dataset_size	small / medium / large / very_large	1.2	Total volume of data the system processes.
query_volume	low / medium / high / very_high	1.0	Expected request throughput (QPS).
latency_requirement	relaxed / moderate / strict / ultra_low	1.1	Response time SLA.
data_volatility	static / low / moderate / high	1.3	How often the underlying data changes.
accuracy_requirement	moderate / high / very_high / critical	1.2	Tolerance for hallucinations and wrong answers.
domain_specificity	general / moderate / specialized / highly_specialized	1.1	How much domain expertise the model must encode.
security_level	standard / elevated / high / critical	0.9	Data classification and compliance burden.
cost_sensitivity	low / moderate / high / very_high	0.8	How strict the budget is.
deployment_preference	cloud / on_premise / hybrid / edge	0.7	Where the system is physically deployed.
user_scale	small / medium / large / enterprise	0.8	Headcount of end users.

6. Signal Extraction - Three-Layer Strategy

File: `backend/services/signal_extractor.py`. Extraction runs in three layers; each layer fills in what previous layers could not produce. The LLM is the primary source; the keyword scan boosts confidence when both agree; the heuristic regex layer takes over when the LLM is unavailable.

6.1 Layer 1 - Keyword Pre-Extraction (zero cost)

For each signal, regex-scan the document for category keywords (dataset, latency, accuracy, etc.). Captures the surrounding sentence as `source_text`. No value is assigned at this layer - only evidence that the topic is mentioned. Confidence is capped at 0.3 so the LLM can always override.

6.2 Layer 2 - LLM Extraction (primary)

A single prompt asks the LLM (Ollama or OpenAI) to fill all 10 signals as JSON. The prompt:

- Lists allowed values verbatim - the LLM cannot invent new ones.
- Maps natural-language phrases to canonical values ("under 2 seconds" -> relaxed; "HIPAA" -> critical security).
- Demands a `source_text` quote per signal - we verify it appears in the document.
- Reserves `critical` for regulated clinical / financial / safety-of-life contexts, and `highly_specialized` for proprietary vocabularies. Internal company knowledge is specialized, not `highly_specialized`.

Why prompt tightening matters

Before this constraint, the LLM was over-classifying any compliance-flavoured doc as `accuracy=critical` and any industry-specific doc as `domain=highly_specialized`. Both values swing the scoring engine sharply toward FineTuning - this is exactly how systemic bias was being introduced.

6.3 Layer 3 - Heuristic Regex Fallback

If the LLM call fails (timeout, malformed JSON, Ollama not running), a pure-regex extractor takes over. It contains carefully-tuned patterns for each of the 10 signals, including numeric resolvers - "500 GB" -> large, "800 queries per minute" -> high QPS, "200 templates" -> small. The regex layer also includes a universal negation guard (see section 13.5) so phrases like "no real-time streaming requirement" do not match as if they meant the opposite.

Chunked extraction for large documents

If the post-filter context exceeds 8,000 characters, we split into 5,000-char overlapping chunks (500-char overlap) and extract from each chunk concurrently (Semaphore bound of 5). The winning signal per name is the one with the highest confidence across chunks. This is why the 13-section stress test (doc 10) does not lose signals at the tail.

Caching

Extraction results are cached in memory keyed by `f"p:{prompt_fingerprint}\x00{full_text}"`. Re-uploading the same document skips

the LLM entirely. The prompt fingerprint auto-invalidates the cache whenever the extraction prompt changes - no manual flush required.

6.4 The Actual LLM Prompt

The prompt is the most important file in the entire pipeline. Every value below is a hard constraint the LLM must honour.

```
You are an expert AI architecture analyst.
Read the document and extract 10 architecture signals as JSON.

SIGNAL NAMES AND ALLOWED VALUES - use EXACTLY these strings:
dataset_size: small | medium | large | very_large
query_volume: low | medium | high | very_high
latency_requirement: relaxed | moderate | strict | ultra_low
data_volatility: static | low | moderate | high
accuracy_requirement: moderate | high | very_high | critical
domain_specificity: general | moderate | specialized | highly_specialized
security_level: standard | elevated | high | critical
cost_sensitivity: low | moderate | high | very_high
deployment_preference: cloud | on_premise | hybrid | edge
user_scale: small | medium | large | enterprise

CONFIDENCE SCALE:
0.0 = signal not mentioned at all (value = null)
0.3 = weakly implied
0.5 = moderately clear
0.7 = clearly stated
0.9 = explicitly defined with numbers / keywords

IMPORTANT RULES:
1. For "value": null ONLY when document gives zero signal.
2. For "source_text": one exact sentence from the document.
3. Set confidence >= 0.3 whenever you can infer a value.
4. Output raw JSON only - no markdown, no code blocks.
```

7. Vector Indexing and Retrieval

File: `backend/app/services/vector_service.py`. The document is chunked, embedded, and indexed in FAISS so the frontend can show source quotes and follow-up questions can retrieve additional context. Indexing is best-effort - if it fails the rest of the pipeline continues without semantic retrieval.

- **Chunk size:** approximately 512 tokens with 50-token overlap.
- **Embedding model:** sentence-transformers/all-MiniLM-L6-v2 - 384-dim, fast, runs locally.
- **Index:** FAISS IndexFlatL2 keyed by `session_id`.
- **Failure mode:** if FAISS or the embedder is unavailable, the stage logs a warning, marks the trace step as 'skipped', and continues - a recommendation is still produced.

8. Scoring Engine - Rules, Weights, Synergies

File: backend/services/scoring_engine.py. This is the deterministic core. Given a set of 10 signals (values plus confidences), it always produces the same scores, ranking, and recommendation. No randomness, no temperature, no LLM call.

8.1 Per-signal score lookup

For each signal, look up the value in SCORING_RULES. The lookup returns a per-architecture score in [0.0, 1.0]:

```
SCORING_RULES["data_volatility"]["high"]
# {"RAG": 1.0, "FineTuning": 0.2, "CAG": 0.1, "Hybrid": 0.5}

SCORING_RULES["accuracy_requirement"]["very_high"]
# {"RAG": 0.9, "FineTuning": 0.8, "CAG": 0.4, "Hybrid": 0.7}
```

8.2 Weighted sum and normalisation

```
effective_weight = SIGNAL_WEIGHTS[signal] * confidence
for each architecture:
    arch_total += rule_score * effective_weight
    total_weight += effective_weight

# Normalise to a 0-100 percentage:
score[arch] = (arch_total / total_weight) * 100
```

Confidence acts as a per-signal damper: a low-confidence signal contributes proportionally less to the final scores. Missing signals (value = null) are skipped entirely.

8.3 Hybrid synergy bonus

Pure rule-based scoring cannot produce Hybrid as the winner because Hybrid's per-signal score is roughly the midpoint of RAG and FineTuning. We detect genuine cross-architecture tension from the signals themselves and apply a bonus only when both dimensions are strongly evidenced. The procedure:

- Count signals with RAG-score ≥ 0.8 (strong RAG pulls).
- Count signals with FT-score ≥ 0.8 (strong FT pulls).
- Compute distinct sets - rag_only and ft_only - by excluding signals strong on both sides.
- Require at least 2 distinct signals on each side.
- Require at least 1 maximal RAG pull (score ≥ 0.95) - the FT-killer signal. Only data_volatility=high or dataset_size=very_large achieve this.
- Bonus = $\min(\text{tension} \times 8.0 \times \text{avg_weight}, 22.0)$. Added to Hybrid's score before final ranking.

Why the maximal-pull requirement

Without it, a doc with volatility=moderate + dataset=large + specialized domain triggered Hybrid (RAG had 2 strong pulls, FT had 2). But moderate volatility does not require real retrieval - periodic FT retraining handles it. The 0.95 threshold is reached only by signals that genuinely break a single-arch design.

8.4 CAG synergy bonus

CAG's sweet spot is a small, bounded, slow-changing corpus that fits in an LLM context window. The per-signal rules under-value CAG when those structural conditions coexist with a specialized domain or very_high accuracy - because those signals strongly favour FT independently. A bonus of +14 is applied when:

```
dataset_size == "small"
AND data_volatility IN ("static", "low")
AND query_volume IN ("low", "medium")
```

8.5 Ranking and why-not

Architectures are sorted by score (descending). The top one is the recommendation. Each architecture gets a suitability label based on its absolute score:

- ≥ 75 - Highly Suitable
- 55 to 74 - Suitable
- 40 to 54 - Moderately Suitable
- < 40 - Not Recommended

For each non-winning architecture, a why-not blurb is generated by identifying its three weakest factors and reporting the score gap versus the winner.

8.6 Rules Table Highlights

Full table lives in SCORING_RULES. Below are the cells with the largest discriminating power - the rows that effectively decide most documents.

Signal = Value	RAG	FT	CAG	Hybrid	Why it matters
data_volatility = high	1.00	0.20	0.10	0.50	RAG-only winner
data_volatility = static	0.50	0.90	0.80	0.40	FT and CAG friendly
dataset_size = very_large	1.00	0.30	0.10	0.70	RAG wins on scale
dataset_size = small	0.30	0.60	0.90	0.30	CAG wins on size
accuracy = critical	0.78	0.95	0.20	0.70	FT strong; RAG raised after rebalance
accuracy = very_high	0.90	0.80	0.40	0.70	RAG with grounding
domain = highly_specialized	0.68	0.95	0.25	0.65	FT strong; RAG raised
domain = specialized	0.85	0.75	0.45	0.60	RAG retrieves a domain corpus
latency = ultra_low	0.20	1.00	0.20	0.40	Retrieval hop kills RAG
deployment = hybrid	0.70	0.60	0.40	0.75	Hybrid edge

9. Architecture Recommendations and Confidence

RAG - Retrieval-Augmented Generation

Combine a retrieval system (FAISS or vector DB) with a generative LLM. The model is fixed; knowledge lives outside it.

Strengths

Dynamic data handling, no retraining needed, transparent sources, cost-effective updates.

Weaknesses

Higher latency (retrieval hop), retrieval-quality dependency, context window limits.

Fine-Tuning

Customise a pre-trained LLM on domain-specific data. Knowledge lives inside the model weights.

Strengths

High accuracy, low inference latency, deep domain knowledge, consistent outputs.

Weaknesses

Expensive retraining, data staleness risk, requires labeled training data.

CAG - Context-Augmented Generation

Stuff the entire corpus into the prompt. No vector DB; no training. Works only when the corpus fits in the context window.

Strengths

Simple architecture, low infrastructure cost, fast setup, no vector DB needed.

Weaknesses

Context window limits, not scalable, higher per-query cost for large contexts.

Hybrid - RAG plus Fine-Tuning

Fine-tune a model on the domain AND retrieve from a fresh corpus. Used when neither alone suffices.

Strengths

Maximum flexibility, best accuracy potential, handles edge cases, scalable.

Weaknesses

Complex architecture, higher cost, more maintenance, longer development time.

9.5 Confidence score

Reported as a single number in $[0, 1]$. Combines two factors: (a) the average per-signal confidence across signals that actually have a value, weighted 0.7; and (b) coverage - the

fraction of the 10 signals with a non-null value, weighted 0.3. So a recommendation made from 9 highly-confident signals beats one from 3 highly-confident signals.

```
confidence = 0.7 * avg_active + 0.3 * coverage
```

10. Sensitivity Analysis and Stability

After the main scoring, the engine perturbs each signal one at a time (trying every other allowed value) and re-scores. If any perturbation changes the recommendation, that is flagged as an instability.

Stability score = $1 - (\text{instabilities} / \text{total_perturbations})$. If above 0.7, the recommendation is considered stable. Otherwise the response includes a warning ('Recommendation may change with different input values').

Performance note

Sensitivity runs 30+ `score()` calls per request - expensive. We cache results by hashing the signals dict (md5) so re-scoring the same signals is instant.

11. Persistence - Postgres and Redis

Postgres tables: Project, Session, Signal, Result. **Redis:** 24-hour cache of the AnalysisResponse keyed by session_id.

Project

id, user_id, name, description, analysis_id, mode, status, timestamps.

Session

id, project_id, status, provider, filename, timestamps. No user_id - sessions are anonymous-capable.

Signal

session_id, name, value, confidence, source_text, page_number, source_verified.

Result

session_id, recommended_architecture, scores, ranking, factor_breakdown, why_not, suitability, sensitivity, followup_questions, decision_trace, architecture_details.

12. Decision Trace and Observability

Every analysis carries a `decision_trace` - a chronological list of stages with status (`in_progress` / `complete` / `skipped` / `rejected` / `warning` / `error`), timestamps, and human-readable details. The frontend surfaces this verbatim in the Decision Trace tab so the user can see exactly what happened to their document.

```
[
{"step": "upload", "status": "complete", "timestamp": "..."},
{"step": "parse", "status": "complete",
"details": "Extracted 1284 words from 12 pages"},
{"step": "relevance_check", "status": "complete",
"details": "Document passes relevance check (1284 words)"},
{"step": "section_detection", "status": "complete",
"details": "Detected 8 document sections"},
{"step": "vector_indexing", "status": "complete",
"details": "Indexed 47 text chunks"},
{"step": "signal_extraction", "status": "complete",
"details": "Extracted 9/10 signals"},
{"step": "missing_signals", "status": "warning",
"details": "1 signal missing: cost sensitivity"},
{"step": "scoring", "status": "complete"},
{"step": "recommend", "status": "complete"}
]
```

13. The Journey - Bugs Found and Fixes Applied

This section catalogs every meaningful bug encountered while calibrating the pipeline, why each bug happened, and the exact fix that landed. Read this if you are touching the scoring or the extractor - the answers to "why is this code so specific" live here.

13.1 Guest uploads returned a generic failure

Symptom

Unauthenticated user uploads a valid PDF and the frontend shows the generic 'Upload failed' card.

Root cause

upload.py had `if not uid: raise HTTPException(401)` at the top of the handler. The frontend caught the 401 and rendered the generic failure card because the error did not match any of the `parseUploadError` keywords.

Fix

Removed the auth gate. Sessions have no `user_id` field (already anonymous-capable). Guest projects were already supported on the projects and analysis endpoints; upload was the only inconsistent one.

13.2 Hybrid was mathematically un-winnable

Symptom

The canonical hybrid-candidate document (real-time market feeds + static research corpus + specialized finance domain) was recommended as FineTuning, not Hybrid.

Root cause

Hybrid's per-signal scores in `SCORING_RULES` were roughly the average of RAG and FT scores. When either single arch pulled strongly on any axis, Hybrid was locked into 2nd or 3rd place. There was no mechanism to detect that a doc needed both dimensions.

Fix

Added a synergy-detection pass after base scoring. Counts signals with strong RAG-only pulls vs strong FT-only pulls. When both sides have ≥ 2 distinct strong pulls AND at least one RAG pull is maximal (≥ 0.95 , only volatility=high or dataset=very_large achieve this), Hybrid gets a weighted bonus. Pure single-arch docs (no opposing pulls) get zero bonus and win cleanly.

13.3 CAG was losing to FineTuning on bounded corpora

Symptom

The legal-contract-templates document (200 templates, 800 pages, fits in 400k tokens, updated annually) was recommended as FineTuning instead of CAG.

Root cause

The accuracy=very_high and domain=specialized cells strongly favoured FT, drowning out CAG's wins on dataset_size=small and data_volatility=static. The per-signal rules could not represent the structural fact that 'the whole corpus fits in the prompt' makes CAG strictly correct.

Fix

Added a CAG synergy bonus (+14 points) that fires when dataset_size=small AND data_volatility in {static, low} AND query_volume in {low, medium}. Those three signals together uniquely describe a bounded-corpus situation where CAG should win regardless of how specialized the domain is.

13.4 RAG was penalised on its core strength**Symptom**

Documents that explicitly required cited or grounded responses (textbook RAG) were recommended as FineTuning.

Root cause

accuracy_requirement=very_high gave RAG 0.7 vs FT 0.9. But grounded retrieval with citations is exactly the RAG guarantee - RAG should match or beat FT on accuracy=very_high. Similarly, domain=specialized gave RAG 0.6 vs FT 0.9, even though retrieval over a domain corpus is exactly what RAG does.

Fix

Rebalanced two rows: accuracy_requirement and domain_specificity. RAG bumped to 0.85-0.9 for very_high and 0.78 for critical. domain=specialized: RAG 0.6 -> 0.85. domain=highly_specialized: 0.4 -> 0.68. CAG also bumped modestly so it stops being penalised on accuracy and domain.

13.5 Heuristic extractor was negation-blind**Symptom**

Doc 02 (a static medical archive) was extracted as data_volatility=high. Doc 03 (low-volume legal tool) was extracted as query_volume=high.

Root cause

The doc 02 phrasing 'no real-time streaming requirement' contains the substring 'real-time streaming' which matches the high-volatility regex. The doc 03 phrasing 'no expectation of high concurrency' matches the high-concurrency regex. Neither regex examined what came before the match.

Fix

Added `_is_negated()` that scans the preceding 60 characters of every regex match for negation words (no, not, never, without, zero, cannot, won't). When any appears, the match is skipped and the next candidate is considered. Universal - applies to every signal pattern.

13.6 Volatility regex missed plural 'feeds'**Symptom**

Doc 04 (financial trading platform with explicit 'Real-time feeds:' section) was extracted as data_volatility=static.

Root cause

The high-volatility regex was `real-time (streaming|data|feed|updates?)`. 'feed' matched but 'feeds' did not. 'Real-time feeds' fell through to the static rule, which matched 'static corpus' in the same doc.

Fix

Changed feed to feeds?. Also added market data and a dedicated streamed? data rule.

13.7 Bounded-corpus docs had null dataset_size

Symptom

Doc 03 mentioned '200 templates', '800 pages', 'fits in 400,000 tokens' - none matched any dataset_size pattern. The CAG bonus (which requires dataset_size=small) silently failed.

Root cause

Original regexes only matched units like GB, TB, 'millions of records', etc. Domain-specific quantities went unrecognised.

Fix

Added three new dataset_size patterns: 'N templates / contracts / playbooks' -> small, 'N pages' -> small / medium / large via a numeric resolver, 'fits in context' / 'bounded corpus' -> small.

13.8 Word-order mismatch on 'queries per day'

Symptom

Doc 03 'roughly 50 to 100 queries per day' -> query_volume = null.

Root cause

The regex expected ordering 'NUM (daily|per day) NOUN' - number, then period, then noun. Doc 03's natural order was reversed: number, then noun, then 'per day'.

Fix

Added a second pattern handling the natural ordering. Also added 'queries per minute' with a 1/60 QPS converter (doc 04 says '800 queries per minute').

13.9 Professional-headcount scale was unmatched

Symptom

Doc 02's 'radiologists', doc 03's 'associates', doc 04's 'portfolio managers' - none matched the user_scale regex. Result: user_scale=null for the docs that most clearly stated their user base.

Root cause

The regex only matched 'users / employees / people' literally. Industry-specific role names were unrecognised.

Fix

Added `\b(\d+)\s+(analysts|associates|radiologists|agents|managers|specialists|consultants|operators|employees)\b` - reuses the existing NUM_USERS numeric resolver.

13.10 Hybrid bonus calibration drift

Symptom

After rebalancing RAG upward (13.4), Hybrid started losing to RAG by 1-3 points on doc 04 and doc 10 - even though the synergy detector was firing correctly.

Root cause

The rebalanced RAG rules pushed the base RAG score higher; the synergy multiplier (originally 6.5) was no longer strong enough to overtake.

Fix

Bumped Hybrid bonus multiplier 6.5 -> 8.0, with cap raised 20 -> 22. Does not affect single-arch docs (they do not trigger the bonus at all), but gives hybrid cases enough force to overtake the now-stronger RAG score.

14. Test Coverage and Verification

Two layers of verification cover the pipeline:

14.1 Layer A - Scoring engine unit tests

backend/verify_scoring.py constructs hand-derived signal sets for each test doc (including 'aggressive LLM extraction' variants) and asserts the scoring engine produces the expected architecture. Catches regressions in rules, weights, and synergy bonuses.

14.2 Layer B - Real-doc heuristic tests

backend/diagnose_pipeline.py reads each test doc's actual markdown content, runs the actual heuristic extractor on it, then scores. This is the only way to catch extractor bugs (negation, missing patterns, word-order). Bypasses Ollama entirely - purely deterministic.

14.3 Test document matrix

Test doc	Expected	What it tests
01_perfect_rag_candidate	RAG	Internal KB, 80GB, weekly updates, on-prem.
02_perfect_finetuning_candidate	FineTuning	Medical, 1.2TB static, HIPAA, FDA.
03_perfect_cag_candidate	CAG	200 legal templates, 800 pages, annual updates.
04_perfect_hybrid_candidate	Hybrid	Trading: static research + real-time feeds.
05_edge_case_ambiguous_signals	(any)	Tests low-confidence handling.
06_edge_case_minimal_valid	(any)	Barely passes word and keyword gates.
07_rejection_too_short	REJECT	Below 80 words.
08_rejection_wrong_doc_type	REJECT	Cost report; fails keyword gate.
09_rejection_archguide_report	REJECT	Circular upload detection.
10_stress_test_large_doc	Hybrid	13 sections; tests chunked extraction.
11_rejection_low_density	REJECT	Newsletter; mentions data but no requirements.

14.4 Current status

All 5 architecture-recommendation cases (01, 02, 03, 04, 10) pass with the actual heuristic extractor on the actual doc text - no LLM, no hand-coded signals. The 4 rejection cases (07, 08, 09, 11) never reach the scoring engine; they are blocked at the word-count, keyword-coverage, or circular-upload gate.

15. Operational Cheat Sheet

15.1 Run the diagnostic

```
cd backend
python diagnose_pipeline.py # heuristic extractor + scoring
python verify_scoring.py # scoring engine alone
```

15.2 Invalidate the extraction cache

The cache is keyed by the prompt fingerprint. Any change to `EXTRACTION_PROMPT` automatically invalidates the cache. To force flush manually, restart the backend process.

15.3 Add a new signal

- Add the entry to `SIGNAL_SCHEMA` with description and keywords.
- Add an `EXTRACTION_PROMPT` mapping rule ('phrase -> value').
- Add an entry to `SIGNAL_OPTIONS` for the frontend questionnaire.
- Add a row to `SCORING_RULES` with per-arch scores for every allowed value.
- Add an entry to `SIGNAL_WEIGHTS`.
- Add a heuristic block to `_HEURISTIC_RULES`.
- Re-run `verify_scoring.py` and `diagnose_pipeline.py`.

15.4 Common failure modes

- **Ollama unreachable:** heuristic fallback kicks in. Check `ollama serve` and `ollama list`.
- **422 Insufficient signal confidence:** doc passed keyword gates but only 0-2 high-confidence signals extracted. Usually means the doc is too vague.
- **FAISS indexing skipped in trace:** embedder not loaded; recommendation still produced.
- **Signal concentration warning:** all signals from a single page. Doc passed gates but content is concentrated. Recommendation is correct but coverage is thin.